

AQI Predictor

Project development, modelling, deployment and evaluation summary

Executive Summary

AQI Predictor is a machine-learning forecasting system developed for Gujranwala, Punjab, Pakistan. The project combines historical weather observations and air-quality variables with temporal and AQI-history features to forecast US AQI at 24-, 48- and 72-hour horizons.

The modelling work progressed from a Random Forest baseline to XGBoost and then CatBoost. The experiments show that CatBoost produced the strongest overall results on the evaluated horizons, with the best 24-hour tuned configuration reaching an R^2 of 0.6979 and RMSE of 19.37. Separate CatBoost models were retained for each forecast horizon.

The final application is deployed as a Streamlit dashboard. Models are stored and retrieved through Hopsworks, recent data is fetched from Open-Meteo, and SHAP is integrated into the dashboard to explain the features contributing to each forecast.

Project at a Glance

Item	Implementation
Location	Gujranwala, Punjab, Pakistan (32.1877, 74.1945)
Historical period	1 Jan 2024 – 31 Jul 2026
Forecast horizons	24, 48 and 72 hours
Main target	Future US AQI (us_aqi shifted forward by horizon)
Final algorithm	CatBoost Regressor
Explainability	SHAP TreeExplainer
Data / feature store	Hopsworks
Data APIs	Open-Meteo weather and air-quality APIs
Application	Streamlit dashboard
Automation	GitHub Actions hourly pipeline / backfill workflows

Key Outcome

The project moved from exploratory analysis and baseline modelling to a working production-style pipeline with horizon-specific CatBoost models, automated data processing, model retrieval, an interactive dashboard and model explainability.

1. Data Collection and Preparation

Location and observation window

The notebook defines Gujranwala at latitude 32.1877 and longitude 74.1945, with Asia/Karachi as the working timezone. Historical data was collected from 2024-01-01 through 2026-07-31.

The weather archive returned 22,632 hourly observations and six weather variables. Air-quality data was collected for the same period and combined with the weather data by timestamp.

Variables used

Group	Variables
Weather	temperature_2m, relative_humidity_2m, wind_speed_10m, surface_pressure, precipitation, cloud_cover
Pollutants	pm10, pm2_5, carbon_monoxide, nitrogen_dioxide, sulphur_dioxide, ozone
AQI	us_aqi
Temporal	month, hour, day_of_week
History / engineered	aqi_lag_1, aqi_lag_6, aqi_lag_12, aqi_lag_24, aqi_rolling_6, aqi_rolling_12, aqi_rolling_24

Observed data characteristics

The merged dataset contained 22,632 rows before the lag/target transformations. US AQI ranged from 57 to 355, with a mean of 143.82 and standard deviation of 47.04. PM2.5 ranged from 2.2 to 348.3, while PM10 ranged from 3.1 to 763.7.

Variable	Mean	Min	Max
Temperature (°C)	23.62	2.6	45.5
Relative humidity (%)	63.67	7	100
Wind speed	7.15	0	36.3
PM10	90.65	3.1	763.7
PM2.5	63.78	2.2	348.3
Carbon monoxide	722.43	120	3837
NO ₂	20.71	0	130.4
SO ₂	13.52	0	75.5
Ozone	96.36	0	255
US AQI	143.82	57	355

The notebook also inspected the AQI distribution and correlation structure. PM2.5 had the strongest simple correlation with current US AQI (0.771), followed by carbon monoxide (0.636), sulphur dioxide (0.631), and PM10 (0.621). Wind speed (-0.304) and temperature (-0.392) showed negative correlations.

These correlations were used as exploratory evidence rather than as a substitute for model-based feature selection.

2. Feature Engineering and Forecast Design

Temporal structure

The data was ordered chronologically and temporal fields were retained to represent recurring hourly and weekly patterns. The modelling experiments used month, hour and day_of_week.

AQI history features

To capture short-term persistence, the project created lagged AQI variables at 1, 6, 12 and 24 hours. Rolling means over 6, 12 and 24 hours were also created. These features allow the model to use recent AQI level and trend information rather than relying only on instantaneous pollutant readings.

Feature	Purpose
aqi_lag_1	Most recent previous AQI
aqi_lag_6	AQI six hours earlier
aqi_lag_12	AQI twelve hours earlier
aqi_lag_24	AQI one day earlier
aqi_rolling_6	Short-term six-hour AQI average
aqi_rolling_12	Twelve-hour AQI average
aqi_rolling_24	Daily AQI average context

Target construction

The 24-hour target was created with `target_aqi = us_aqi.shift(-24)`. Equivalent future targets were then created for 48 and 72 hours using negative shifts. Rows with unavailable lag or target values were removed.

Training / test strategy

The notebook used an 80/20 chronological split: the first 80% of each horizon-specific modelling dataset was used for training and the final 20% for testing. This preserves temporal ordering rather than randomly mixing future observations into the training set.

Final feature set

The core model feature list contained 19 variables: six weather variables, six pollutant variables, current US AQI, four AQI lags, and three rolling AQI statistics. After removing rows with missing lag/target values, the 24-hour modelling dataset contained 22,584 observations.

This feature design became the basis for the Random Forest, XGBoost and CatBoost comparisons, keeping the algorithm comparison focused on the learning method rather than changing the data pipeline between models.

3. Model Experiments

Random Forest baseline

Random Forest was used as the first tree-based baseline with 100 estimators and `random_state=42`. It provided a useful reference for judging the subsequent boosting models.

Horizon	MAE	RMSE	R ²
24 hours	16.164	20.585	0.6587
48 hours	21.593	26.780	0.4224
72 hours	23.346	28.689	0.3374

XGBoost

XGBoost was tested on the same engineered feature set. The initial configuration used 200 estimators, `learning_rate=0.05` and `max_depth=6`. A second feature experiment produced slightly better 24- and 48-hour results.

Experiment	Horizon	MAE	RMSE	R ²
XGBoost baseline	24h	15.398	19.707	0.6872
XGBoost baseline	48h	21.056	26.212	0.4467
XGBoost baseline	72h	22.802	27.795	0.3780
XGBoost feature experiment	24h	15.166	19.720	0.6868
XGBoost feature experiment	48h	20.675	25.658	0.4700

XGBoost configuration tuning

Model	Iterations	Learning rate	Depth	MAE	RMSE	R ²
1	200	0.05	6	15.398	19.707	0.6872
2	300	0.05	6	15.467	19.818	0.6836
3	200	0.03	6	15.379	19.656	0.6888
4	200	0.05	4	15.419	19.794	0.6844
5	200	0.05	8	15.696	20.087	0.6750
6	300	0.03	4	15.427	19.779	0.6849

The best XGBoost configuration in this tuning table was Model 3 (200 estimators, learning rate 0.03, depth 6), with MAE 15.379, RMSE 19.656 and R² 0.6888 on the 24-hour test set.

4. CatBoost Selection and Fine Tuning

Initial CatBoost comparison

CatBoost Regressor was then evaluated using the same overall feature and chronological-split strategy. The initial configuration used 200 iterations, learning_rate=0.05, depth=6 and random_seed=42.

Horizon	MAE	RMSE	R ²
24 hours	15.262	19.481	0.6943
48 hours	20.678	25.338	0.4829
72 hours	22.250	26.964	0.4147

Why CatBoost was finalized

CatBoost gave the strongest overall balance across the three horizons in the tested models. It improved substantially over the Random Forest baseline at 48 and 72 hours and matched or exceeded the tuned XGBoost results in the key 24-hour evaluation.

24-hour CatBoost fine tuning

Model	Iterations	LR	Depth	MAE	RMSE	R ²
Baseline	200	0.05	6	15.262	19.481	0.6943
Model 2	300	0.05	6	15.193	19.432	0.6958
Model 3	500	0.03	6	15.177	19.430	0.6959
Model 4	300	0.03	5	15.476	19.691	0.6877
Model 5	300	0.05	5	15.285	19.533	0.6927
Model 6	300	0.05	7	15.143	19.366	0.6979
Model 7	500	0.03	5	15.338	19.587	0.6910

Model 6 was selected for the 24-hour horizon: 300 iterations, learning rate 0.05 and depth 7. It produced the best tuning result with MAE 15.143, RMSE 19.366 and R² 0.6979.

Final horizon-specific models

Horizon	Iterations	Learning rate	Depth
24h	300	0.05	7
48h	200	0.05	6
72h	200	0.05	6

The notebook's final training cells retrained the selected 24-hour model on the full available 24-hour modelling data. The final 48- and 72-hour models were trained using the CatBoost configurations shown above. The 72-hour model was also tested with a 300-iteration, depth-7 configuration, improving its test result to MAE 21.981, RMSE 26.810 and R² 0.4213.

5. Final Modelling Approach

Architecture

The deployed approach uses three separate CatBoost regression models rather than one multi-output model. Each model learns a different future target: AQI 24, 48 and 72 hours ahead.

Stage	What happens
1. Data ingestion	Recent weather and air-quality observations are fetched from Open-Meteo.
2. Merge	Weather and air-quality records are aligned on datetime.
3. Feature engineering	Temporal, AQI lag and rolling features are generated.
4. Latest row	The most recent complete feature row is selected for forecasting.
5. Model retrieval	The corresponding CatBoost models are downloaded from Hopsworks.
6. Prediction	Each horizon-specific model produces one AQI forecast.
7. Explainability	SHAP TreeExplainer calculates feature-level contributions.
8. Presentation	Streamlit displays forecast cards, trend information and SHAP explanations.

Prediction behaviour

The production prediction pipeline retrieves the latest 48 hours of weather and air-quality observations, merges them, creates the same engineered features used during training, validates that all required features exist and are non-null, and then applies the three stored CatBoost models.

Predictions are constrained to a minimum of zero before being presented. Each result records the prediction creation timestamp, forecast timestamp, forecast horizon and predicted AQI.

Explainability with SHAP

SHAP was added as a project requirement and integrated directly into the prediction flow. A TreeExplainer is created for each CatBoost model and the SHAP values for the latest feature row are returned with the forecast.

The dashboard presents the largest feature contributions for the selected horizon. Positive SHAP values indicate features pushing the prediction higher; negative values indicate features pushing it lower. This provides a model-level explanation rather than presenting SHAP as a separate manual workflow.

Evaluation interpretation

Performance decreases as the forecast horizon increases, which is expected for AQI forecasting because uncertainty grows further into the future. The 24-hour model achieved the strongest test fit, while the 48- and 72-hour models remained useful but had lower R^2 values.

6. Engineering, Deployment and Problems Solved

Backfill and pipeline development

A practical constraint appeared during development: the historical backfill could not be run reliably in the local development environment. The solution was to move the backfill workload into GitHub Actions, allowing the historical data pipeline to run in a controlled CI environment. The backfill was executed manually once to populate the required history before normal scheduled operation.

Import and package structure

The repository uses a src package. Running pipelines with `python -m src.feature_pipeline` exposed imports that were written as top-level modules. Imports such as `data_fetch` and `hopswrksclients` were changed to package-relative imports (for example, `from .data_fetch import ...`).

A related deployment issue occurred when Streamlit could not resolve the src package. The dashboard was updated to place the repository root on `sys.path` before importing the prediction pipeline, matching the deployed repository structure.

Configuration and environment variables

The pipeline initially attempted to import `LATITUDE` and `LONGITUDE` from a config module while the workflow supplied them as environment variables. The configuration was aligned with the package structure so the application could read deployment-time values consistently.

Model download conflict

Repeated forecast generation exposed a model-file collision because a CatBoost model was already present in the download directory. An attempted fix using an overwrite argument failed because the installed Hopswrks model-download method did not accept that keyword. The working approach was to remove the `downloaded_models` directory before creating fresh horizon-specific directories.

Open-Meteo rate limiting

The live application also encountered HTTP 429 responses from the Open-Meteo forecast endpoint. The error occurred while requesting recent data, including a request with missing/default coordinates (0.0, 0.0) during one deployment state. This highlighted both external API rate limiting and the importance of validating deployment configuration before repeated dashboard requests.

Streamlit rendering and return-format issues

Several dashboard iterations exposed HTML rendering problems where raw markup was displayed instead of forecast cards. The final direction was to keep the cards simple and use native Streamlit components for warning labels and status messages. A separate `KeyError` also occurred because the dashboard expected a tuple while the prediction function had been changed to return a dictionary; the interface was then made explicit and compatible with the active return format.

7. Automation, Final Application and Conclusion

GitHub Actions scheduling

The automated pipeline was intended to run hourly. During deployment, the workflow was observed running only every several hours despite having no failed runs. The issue was treated as scheduler/runner behaviour rather than an application failure. GitHub Actions scheduling is not a hard real-time scheduler and scheduled jobs can be delayed or skipped under platform load or scheduling conditions.

The workflow interval was subsequently changed during development to reduce the practical impact of missed hourly executions. Importantly, this was distinguished from a pipeline failure: the workflow could remain green while still not executing at the intended wall-clock frequency.

Final dashboard

The final dashboard is designed around a simple forecast-first experience. It identifies Gujranwala as the forecast location, provides 24-, 48- and 72-hour forecast cards, classifies each AQI value with health-oriented labels, displays a forecast trend chart, provides a health outlook, and exposes SHAP explanations for each horizon.

Final technology stack

Layer	Technology / component
Language	Python
Data handling	Pandas, NumPy
Machine learning	CatBoost Regressor
Compared models	Random Forest Regressor, XGBoost Regressor
Explainability	SHAP
Data source	Open-Meteo Weather and Air Quality APIs
Feature/model platform	Hopsworks
Dashboard	Streamlit
Visualisation	Plotly
Automation	GitHub Actions

Final results

Final / evaluated model	MAE	RMSE	R ²
CatBoost 24h tuned	15.143	19.366	0.6979
CatBoost 48h baseline	20.678	25.338	0.4829
CatBoost 72h baseline	22.250	26.964	0.4147
CatBoost 72h tuned experiment	21.981	26.810	0.4213

Conclusion

The project followed an evidence-driven progression: collect and inspect the data, engineer temporal and AQI-history features, establish a Random Forest baseline, compare XGBoost, evaluate CatBoost, tune the strongest candidate, and then operationalise the selected models through Hopsworks, GitHub Actions and Streamlit.

The final system is therefore more than a standalone regression notebook. It is a small end-to-end forecasting application with automated data movement, horizon-specific models, explainability and a user-facing dashboard. The main technical limitation demonstrated by the experiments is the expected decline in predictive performance at longer horizons, while the main operational limitations encountered were external API rate limits and non-real-time scheduled workflow execution.

Source note

Model metrics, dataset dimensions, feature definitions, target construction, model configurations and tuning results in this report were extracted from the supplied AQI_Predictor_EDA.ipynb notebook. Deployment and debugging events are summarized from the project's development history.